

Week 12 Tutorial

Sound in p5.js

In this tutorial we will cover some different ways to use sound in p5.js to animate visuals. These examples cover a variety of different techniques and should illustrate ways that you can use sound in your major assignments.

In all these examples we will load a sound file from disk, you can of course change this file for a file of your choice. For today's work, and to make the code in this tutorial work, please download the sound file we will be working with for all examples.

You can download the file [here](https://canvas.sydney.edu.au/courses/72128/files/47943584?wrap=1). (<https://canvas.sydney.edu.au/courses/72128/files/47943584?wrap=1>)  (https://canvas.sydney.edu.au/courses/72128/files/47943584/download?download_frd=1)

****Make sure to unzip the file after you download it****

It comes from Freesound - an online collection of 630,472 free, open source sounds, and we are working with this file today:

<https://freesound.org/people/multitonbits/sounds/383935/>

The first thing to note about sound and p5.js is that we cannot just play sound without user interaction. In the old days of the internet, websites would just start blasting sound when you loaded them. As you can imagine this was an awful experience. Now browsers require the user to have some kind of interaction before a sound can play, so you will see all our examples use a button to start the file playing. This can be other kinds of user interaction as well, such as clicking on the canvas.

Part 1 Loading a sound file and playback:

Objective:

In this section, we will explore how to load and play a sound file in p5.js. This example demonstrates the basic setup required to work with audio, including handling user interaction to initiate playback.

Challenges:

- We need to load an audio file and prepare it for playback.
- We must create user interaction to comply with browser restrictions on auto playing sound.
- We need to visually represent the sound amplitude in real-time.

Solutions:

- We will preload the audio file to ensure it is fully loaded before attempting to play it.
- A play/pause button will be added to control the sound playback, allowing us to meet browser autoplay policies.
- We will use an amplitude analyzer to measure and display the volume of the audio as an ellipse whose size changes with the sound level.

Implementation:

```
// We will create two variables: song and analyser
// song will hold the audio data we load from disk
// analyser will hold the amplitude data from the audio
let song, analyser;

function preload() {
  // Audio file from freesound https://freesound.org/people/multitonbits/sounds/383935/?
  // licensed under the Creative Commons 0 License
  // Let's load the sound file in preload
  song = loadSound('assets/383935__multitonbits__bs_electricity-bass-2.wav');
}

function setup() {
  createCanvas(400, 400);

  // Create a new Amplitude analyser, this will analyse the volume of the song
  analyser = new p5.Amplitude();

  // Connect the input of the analyser to the song
  analyser.setInput(song);

  // Add a button for play/pause
  // We cannot play sound automatically in p5.js, so we need to add a button to start the sound
  let button = createButton('Play/Pause');

  // Set the position of the button to the bottom centre
  button.position((width - button.width) / 2, height - button.height - 2);

  // We set the action of the button by choosing what action and then a function to run
  // In this case, we want to run the function play_pause when the button is pressed
  button.mousePressed(play_pause);
}

function draw() {
  background(0);

  // Get the average (root mean square) amplitude
  let rms = analyser.getLevel();
  fill(127);
  // Draw an ellipse with size based on volume
  ellipse(width / 2, height / 2, 10 + rms * 200, 10 + rms * 200);
}

function play_pause() {
  if (song.isPlaying()) {
    song.stop();
  } else {
    // We can use song.play() here if we want the song to play once
    // In this case, we want the song to loop, so we call song.loop()
    song.loop();
  }
}
```

Part 2 Setting volume and pan:

Objective:

In this section, we will create a sketch that allows dynamic control of audio properties such as volume and stereo panning based on user interaction. These properties will be controlled by mouse movements.

Challenges:

- Real-time adjustment of audio volume and stereo panning using mouse coordinates.
- Displaying the current settings of volume and pan on the canvas.

Solutions:

- The `mouseMoved()` function will map the mouse coordinates to volume and pan values, providing a responsive audio control.
- Current values of volume and pan will be continuously displayed on the screen.

Implementation:

```
// We will create two variables: song and analyser
// song will hold the audio data we load from disk
// analyser will hold the amplitude data from the audio
let song, analyser;

// Let's make variables for the volume and pan
// We will start the volume at 1.0 (full volume)
let volume = 1.0;
// We will start the pan at 0.0 (centre). It goes from -1.0 (left only) to 1.0 (right only)
let pan = 0.0;

function preload() {
  // Audio file from freesound https://freesound.org/people/multitonbits/sounds/383935/?
  // licensed under the Creative Commons 0 License
  // Let's load the sound file in preload
  song = loadSound('assets/383935__multitonbits__bs_electricity-bass-2.wav');
}

function setup() {
  createCanvas(400, 400);

  // Create a new Amplitude analyser, this will analyse the volume of the song
  analyser = new p5.Amplitude();

  // Connect the input of the analyser to the song
  analyser.setInput(song);

  // Add a button for play/pause
  // We cannot play sound automatically in p5.js, so we need to add a button to start the sound
  let button = createButton('Play/Pause');

  // Set the position of the button to the bottom centre
  button.position((width - button.width) / 2, height - button.height - 2);

  // We set the action of the button by choosing what action and then a function to run
```

```

// In this case, we want to run the function play_pause when the button is pressed
button.mousePressed(play_pause);
}

function draw() {
  background(0);

  // Get the average (root mean square) amplitude
  let rms = analyser.getLevel();
  fill(127);
  // Draw an ellipse with size based on volume
  ellipse(width / 2, height / 2, 10 + rms * 200, 10 + rms * 200);

  // Draw the volume value on the screen
  fill(255);
  // We will limit the number of decimal places to 2 using the function toFixed(2)
  text('Volume: ' + volume.toFixed(2), 10, 20);

  // Draw the pan value on the screen
  text('Pan: ' + pan.toFixed(2), 10, 40);
}

function play_pause() {
  if (song.isPlaying()) {
    song.stop();
  } else {
    // We can use song.play() here if we want the song to play once
    // In this case, we want the song to loop, so we call song.loop()
    song.loop();
  }
}

function mouseMoved() {
  // Map the mouseY to a volume value between 0 and 1
  volume = map(mouseY, 0, height, 1, 0);
  song.setVolume(volume);

  // Map the mouseX to a pan value between -1 and 1
  pan = map(mouseX, 0, width, -1, 1);
  song.pan(pan);
}

```

Part 3 Simple FFT:

Objective:

In this section, we will create a sketch to analyze and visualize the frequency spectrum of an audio file using Fast Fourier Transform (FFT). This analysis will help illustrate the distribution of different frequencies within the sound.

Challenges:

- Implementing FFT to get detailed frequency data from the audio.
- Visually representing this frequency data as a series of vertical bars.
- Making the representation react to changes in the size of the screen.

Solutions:

- The p5.FFT object will be utilized to conduct a frequency analysis on the audio signal.
- We will draw each frequency band as a rectangle, with its height corresponding to the amplitude of that particular frequency.
- We will colour the rectangles based on their position in the fft spectrum array, using HSB colour mode.
- We will use the width of screen and the number of frequency bins to control the width of the rectangles so they always fill the screen.

Implementation:

```
// Let's make a variable to hold the audio file
let song;

// Let's make a variable to hold the FFT object
let fft;

// Let's make a variable for the number of bins in the FFT object
// This is how many frequency bands we will have
// The number of bins must be a power of 2 between 16 and 1024
// Try changing this value
let numBins = 128;

// We will also have a variable for the smoothing of the FFT
// This averages the values of the frequency bands over time so it doesn't jump around too much
// Smoothing can be a value between 0 and 1
// Try changing this value
let smoothing = 0.8;

// This time we will make a global variable for the button so we can access it in the windowResized function
let button;

// Load sound file before setup() function runs
function preload() {
  // Audio file from freesound https://freesound.org/people/multitonbits/sounds/383935/
  song = loadSound("assets/383935__multitonbits__bs_electricity-bass-2.wav");
}

function setup() {
  createCanvas(windowWidth, windowHeight);

  // Create a new instance of p5.FFT() object
  fft = new p5.FFT(smoothing, numBins);

  song.connect(fft);

  // Add a button for play/pause
  // We cannot play sound automatically in p5.js, so we need to add a button to start the sound
  button = createButton("Play/Pause");

  // Set the position of the button to the bottom centre
  button.position((width - button.width) / 2, height - button.height - 2);

  // We set the action of the button by choosing what action and then a function to run
  // In this case, we want to run the function play_pause when the button is pressed
  button.mousePressed(play_pause);

  // Set the colour mode to HSB
  colorMode(HSB, 255);
}
```

```

function draw() {
  background(0);

  // analyze() method returns an array of amplitude values across the frequency spectrum
  // Amplitude values range between 0 and 255, where at 0, the sound at the specific frequency
band is silent
  // and at 255, the sound at the specific frequency band is at its loudest
  let spectrum = fft.analyze();

  // Calculate width of rectangles
  let size = width / spectrum.length;

  // Loop through the spectrum array to draw a rectangle per frequency band
  for (let i = 0; i < spectrum.length; i++) {
    /*
      We will set the fill color of the rectangle based on its frequency band
      We will use the map function to map the value of i (which frequency band) to the hue val
ue
      The map function takes 5 arguments:
      1. Value to be remapped
      2. Minimum value of the input range
      3. Maximum value of the input range
      4. Minimum value of the output range
      5. Maximum value of the output range
    */
    fill(map(i, 0, spectrum.length, 0, 255), 255, 255);

    // We will use map again to set the x-location of the rectangle based on the frequency ban
d
    // i can be between 0 and the length of the spectrum array
    // We will map this to a value between 0 and the width of the canvas
    let x = map(i, 0, spectrum.length, 0, width);

    // Map the y-location of the rectangle using amplitude at the specific frequency band
    // Now we use the power of each frequency band to set the height of the rectangle
    let y = map(spectrum[i], 0, 255, height, 0);

    // Becuase p5.js draws rectangles from the top left corner, we need to adjust the y value
    // to be height - y
    rect(x, y, size, height - y);
  }
}

function play_pause() {
  if (song.isPlaying()) {
    song.stop();
  } else {
    // We can use song.play() here if we want the song to play once
    // In this case, we want the song to loop, so we call song.loop()
    song.loop();
  }
}

function windowResized() {
  resizeCanvas(windowWidth, windowHeight);

  // Reset the position of the button
  button.position((width - button.width) / 2, height - button.height - 2);
}

```

Part 4 Advanced FFT:

Objective:

In this section, we will create a sketch that uses FFT to analyze the audio spectrum and visualize it in a more engaging way. We will arrange frequency band visualizations around a circle, offering

a radial spectrum analysis. We will also use the spectral centroid and overall amplitude to animate a circle.

Challenges:

- Developing a circular visualization of frequency bands, making the display more visually appealing and informative.
- Keeping the circular display within the boundaries of the screen and reacting to changes in screen size.

Solutions:

- Using the p5.FFT object, we will extract frequency analysis data, focusing on achieving smooth transitions and details in the visualization.
- We will use trigonometry to rotate the canvas to draw each rectangle at the correct rotation.
- The maximum dimensions of the rectangles and the centre circles will be defined as fractions of the smallest screen dimension, to make this sketch responsive.

Implementation:

```
// Let's make a variable to hold the audio file
let song;

// Let's make a variable to hold the FFT object
let fft;

// Let's make a variable for the number of bins in the FFT object
// This is how many frequency bands we will have
// The number of bins must be a power of 2 between 16 and 1024
// Try changing this value
let numBins = 128;

// We will also have a variable for the smoothing of the FFT
// This averages the values of the frequency bands over time so it doesn't jump around too much
// Smoothing can be a value between 0 and 1
// Try changing this value
let smoothing = 0.8;

// This time we will make a global variable for the button so we can access it in the windowResized function
let button;

// Load sound file before setup() function runs
function preload() {
  // Audio file from freesound https://freesound.org/people/multitonbits/sounds/383935/
  song = loadSound("assets/383935__multitonbits__bs_electricity-bass-2.wav");
}

function setup() {
  createCanvas(windowWidth, windowHeight);

  // Create a new instance of p5.FFT() object
  fft = new p5.FFT(smoothing, numBins);

  song.connect(fft);

  // Add a button for play/pause
  // We cannot play sound automatically in p5.js, so we need to add a button to start the sound
```

```

d
button = createButton("Play/Pause");

// Set the position of the button to the bottom centre
button.position((width - button.width) / 2, height - button.height - 2);

// We set the action of the button by choosing what action and then a function to run
// In this case, we want to run the function play_pause when the button is pressed
button.mousePressed(play_pause);

// Set the colour mode to HSB
colorMode(HSB, 255);
}

function draw() {
  background(0);

  // Calculate the minimum dimension to determine the size of the circles
  let minDimension = min(width, height);
  // We will make an inner circle with a radius of 1/5 of the minimum dimension
  let circleRadius = minDimension / 5;
  // We will make the rectangles 2/5 of the minimum dimension so 2 rectangles and the circle will fit in the window
  let maxRectLength = (minDimension * 2) / 5;

  // Get overall amplitude between 20 Hz to 20 kHz
  let amplitude = fft.getEnergy(20, 20000);

  // Get the spectral centroid - this is the "centre of mass" of the frequency spectrum
  let centroidFreq = fft.getCentroid();

  // analyze() method returns an array of amplitude values across the frequency spectrum
  // Amplitude values range between 0 and 255, where at 0, the sound at the specific frequency band is silent
  // and at 255, the sound at the specific frequency band is at its loudest
  let spectrum = fft.analyze();

  // Set the centre for drawing
  translate(width / 2, height / 2);

  for (let i = 0; i < spectrum.length; i++) {
    // We want to spread the rectangles evenly around the circle
    // We will use the map function to map the index of the spectrum array to an angle
    let angle = map(i, 0, spectrum.length, 0, TWO_PI);

    // We use the same calculation as in the simple FFT example to calculate the height of the rectangle
    let rectHeight = map(spectrum[i], 0, 255, 0, maxRectLength);

    // We use push() and pop() to isolate the rotation so each rectangle is rotated individually
    push();
    // Rotate amount is based on which rectangle we are drawing
    rotate(angle);
    // Set the fill colour based on the frequency band as before
    fill(map(i, 0, spectrum.length, 0, 255), 255, 255);
    // Draw the rectangle at 0 on the x-axis, circleRadius on the y-axis
    rect(0, circleRadius, width / spectrum.length, rectHeight);
    // Pop the rotation so the next rectangle is drawn with its own rotation
    pop();
  }

  // We will draw an inner circle that changes size based on the amplitude
  // and changes colour based on the spectral centroid
  // We will map the amplitude to the size of the circle
  let innerCircleSize = map(amplitude, 0, 255, circleRadius / 5, circleRadius);

  // Map the spectral centroid to a colour
  let colorVal = map(centroidFreq, 0, 22050, 0, 255);
  fill(colorVal, 255, 255);

  // Draw the inner circle at the centre of the canvas with a size based on the amplitude

```



```

// We multiply the size by 2 because the size of the ellipse is the diameter, not the radius
ellipse(0, 0, innerCircleSize * 2);
}

function play_pause() {
  if (song.isPlaying()) {
    song.stop();
  } else {
    // We can use song.play() here if we want the song to play once
    // In this case, we want the song to loop, so we call song.loop()
    song.loop();
  }
}

function windowResized() {
  resizeCanvas(windowWidth, windowHeight);

  // Reset the position of the button
  button.position((width - button.width) / 2, height - button.height - 2);
}

```

Part 5 Controlling our shape class with FFT:

Objective:

In this section, we will create a sketch that controls shapes using FFT data. The shapes class has been adapted from last weeks tutorial material, it already creates shapes of random colours and sizes that react to changes in screen size. Now they will change size based on FFT values.

Challenges:

- Manipulating visual properties of shapes (e.g., size and color) in real-time, based on audio frequency data.
- Creating instances of the shape class automatically in a loop without choosing circle or square manually.

Solutions:

- We will alter the original class, removing all the noise components and adding a scale input to the classes draw function.
- We will use a for loop to create instances of the shape class, using a random value to decide if they are squares or circles.
- We will create the same number of shapes as we have FFT bins, this way the value from each bin affects a single shape.

Implementation:

```

// This class will create and manage a random shape with noise
class randomShape {
  // The constructor will take a type of shape as an argument - circle or square
  constructor(type) {
    this.type = type;
  }
}

```

```

    // Randomise the x and y position of the shape, these are limited to 80% of the width and height
    this.x = random(0.8);
    this.y = random(0.8);

    // Randomise the size of the shape between 5% and 20% of the width
    this.size = random(0.05, 0.2);
    // Randomise the colour of the shape
    this.color = [random(255), random(255), random(255)];
    this.scale = 1;
}

display(scale) {
    this.scale = scale;
    // Use the instance colour to fill the shape
    fill(this.color);
    noStroke();
    // Get the smaller dimension of the canvas
    let minDimension = min(width, height);

    // Use the smaller dimension to maintain the aspect ratio
    // Calculate the size of the shape based on the smaller dimension
    // Remember the sizes were made in percentages so this will scale correctly
    let size = this.size * minDimension;

    // We now multiply the size by the scale to make the shapes change size based on the input to the display function
    size = size * this.scale;

    // The x position is the x value of the shape multiplied by the width of the canvas - same idea for the y position
    let x = this.x * width;
    let y = this.y * height;

    // Draw the shape based on the type
    switch (this.type) {
        case "circle":
            ellipse(x, y, size, size);
            break;
        case "square":
            rect(x, y, size, size);
            break;
    }
}

// Create an array to hold the shapes
let shapes = [];

// Let's make a variable to hold the audio file
let song;

// Let's make a variable to hold the FFT object
let fft;

// Let's make a variable for the number of bins in the FFT object
// This is how many frequency bands we will have
// The number of bins must be a power of 2 between 16 and 1024
// Try changing this value
let numBins = 128;

// We will also have a variable for the smoothing of the FFT
// This averages the values of the frequency bands over time so it doesn't jump around too much
// Smoothing can be a value between 0 and 1
// Try changing this value
let smoothing = 0.8;

// This time we will make a global variable for the button so we can access it in the windowResized function
let button;

```

```

// Load sound file before setup() function runs
function preload() {
  // Audio file from freesound https://freesound.org/people/multitonbits/sounds/383935/?song=loadSound\('assets/383935\_\_multitonbits\_\_bs\_electricity-bass-2.wav'\);
}

function setup() {
  createCanvas(windowWidth, windowHeight);

  // Create a new instance of p5.FFT() object
  fft = new p5.FFT(smoothing, numBins);

  song.connect(fft);

  // Add a button for play/pause
  // We cannot play sound automatically in p5.js, so we need to add a button to start the sound
  button = createButton("Play/Pause");

  // Set the position of the button to the bottom centre
  button.position((width - button.width) / 2, height - button.height - 2);

  // We set the action of the button by choosing what action and then a function to run
  // In this case, we want to run the function play_pause when the button is pressed
  button.mousePressed(play_pause);

  // Set the colour mode to HSB
  colorMode(HSB, 255);

  // Let's make a shape for each frequency band
  for (let i = 0; i < numBins; i++) {
    /*
      Randomly choose between 'circle' and 'square'
      This is a ternary operator - it is a shorthand if statement
      It is the same as writing:
      let shapeType = "null";
      if (Math.random() > 0.5) {
        shapeType = "circle";
      } else {
        shapeType = "square";
      }
    */
    let shapeType = Math.random() > 0.5 ? "circle" : "square";
    shapes.push(new randomShape(shapeType));
  }

  // Let's set the rect mode to centre so the squares stay in position but change size
  rectMode(CENTER);
}

function draw() {
  background(0);
  // The analyze() method returns an array of amplitude values across the frequency spectrum
  // Amplitude values range between 0 and 255, where at 0, the sound at the specific frequency
  // band is silent
  // and at 255, the sound at the specific frequency band is at its loudest
  let spectrum = fft.analyze();

  for (let i = 0; i < numBins; i++) {
    // We divide the spectrum values by 255 so they are in the range 0 to 1
    shapes[i].display(spectrum[i]/255);
  }
}

function play_pause() {
  if (song.isPlaying()) {
    song.stop();
  } else {
    // We can use song.play() here if we want the song to play once
    // In this case, we want the song to loop, so we call song.loop()
    song.loop();
  }
}

```

```
}  
  
function windowResized() {  
    resizeCanvas(windowWidth, windowHeight);  
  
    // Reset the position of the button  
    button.position((width - button.width) / 2, height - button.height - 2);  
}
```

Part 6 Inspiration:

The inspiration for this week is the amazing work of Tarik Barri. Tarik Barri is a Dutch audio visual composer based in Berlin. He programmed his own audio-visual software Versum. In this 3D real-time virtual world, he creates audio-visual compositions. He has collaborated with Radiohead, Atoms for Peace, Sote, Nicolas Jaar, Monolake and others.

Tarik works a lot with audio reactive video, and this week I would like to focus on this great collaboration with another amazing artist [Ata 'Sote' Ebtekar](#) ➞

(<https://open.spotify.com/artist/6xOjV520Oih4xkYyk2dqFZ?autoplay=true>) for a work called 'Sacred Horror in design'. You can see a live recording of the work here: <https://vimeo.com/244348082>. ➞ (<https://vimeo.com/244348082>)

There are many more amazing examples of his work on his Vimeo page:

<https://vimeo.com/tarikbarri> ➞ (<https://vimeo.com/tarikbarri>)

Happy coding!

Copyright © The University of Sydney. Unless otherwise indicated, 3rd party material has been reproduced and communicated to you by or on behalf of the University of Sydney in accordance with section 113P of the Copyright Act 1968 (Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.

Live streamed classes in this unit may be recorded to enable students to review the content. If you have concerns about this, please visit our [student guide](#) (<https://canvas.sydney.edu.au/courses/4901/pages/zoom>) and contact the unit coordinator.

[Privacy Statement](#) (<https://www.sydney.edu.au/privacy-statement.html>)